Métodos Formais

Prof. Júlio Machado

Formalizando a Noção de Algoritmo Via Equações Recursivas

1. Introdução

Nosso objetivo é definir uma maneira de formalizar a noção de algoritmo de maneira a permitir, futuramente, a análise e prova de propriedades funcionais sobre estes algoritmos. Essa formalização se dará, inicialmente, na forma de equações recursivas, um modelo matemático adequado aos nossos objetivos.

2. Um primeiro algoritmo

Supondo que a soma de dois valores inteiros é a única operação aritmética que um determinado agente computacional é capaz de executar (lembre-se que a operação de subtração pode ser vista como uma soma de um número negativo), vamos analisar um algoritmo para multiplicar dois números inteiros positivos (de forma equivalente dois números naturais).

Dadas as restrições do agente computacional, uma forma de computar a multiplicação entre dois números inteiros ($m \times n, m \geq 0, n \geq 0$) é a que segue:

5 *3	=	5 * 3 + 0		4 * 4	=	4 * 4 + 0
	=	4 * 3 + 3			=	3 * 4 + 4
	=	3 * 3 + 6			=	2 * 4 + 8
	=	2 * 3 + 9			=	1 * 4 + 12
	=	1 * 3 + 12			=	0 * 4 + 16
	=	0 * 3 + 15			=	16
	=	15				

Note que o exemplo acima permite intuir um algoritmo de multiplicação baseado apenas em somas sucessivas. Esse algoritmo pode ser escrito em língua natural como segue:

- Entrada: $m, n \in \mathbb{N}$
- Saída: $m \times n \in \mathbb{N}$

1. Colocar o valor de *a* igual a zero;
2. Enquanto m > 0, repetir os passos 2.1 e 2.2;
   2.1. Adicionar *n* ao valor de *a*;
   2.2. Subtrair 1 do valor de *m*;
3. A resposta é o valor de *a*;
4. Parar.

Esse algoritmo satisfaz a todas as propriedades que permitem classificar um procedimento como um algoritmo (coleção ordenada de operações, não ambíguas, efetivamente computáveis, produzem um resultado e termina em um tempo finito).

3. Algoritmos especificações equacionais

Existem várias formas de formalizar o processo intuitivo para a multiplicação de dois números apresentado na seção 2. Utilizaremos aqui funções recursivas definidas por equações.
Uma especificação equacional de uma função $f: A \longrightarrow B$ é dada por equações da forma

$$f(x) = e$$

onde a variável *x* (argumento da função) pode ocorrer 0 ou mais vezes na expressão *e*. Se na expressão *e* houver uma ou mais ocorrências de *f*, então *f* é dita recursiva. Funções definidas por equações são avaliadas por substituição e simplificação. Se $a \in A$, então $f(a)$ é avaliada da seguinte forma: substituir todas as ocorrências de *x* em *e* por a. A notação para esta operação é representada como

$$e[x \to a]$$

A utilização de equações recursivas é a maneira pela qual podemos repetir sequências de ações em um laço.

Exemplo: algoritmo de multiplicação formalizado usando equações recursivas

$mult: \mathbb{N} \times \mathbb{N} \to \mathbb{N}$
$mult(\langle m, n \rangle) = multacc(\langle m, n, 0 \rangle)$ $\langle 1 \rangle$
onde
$multacc: \mathbb{N} \times \mathbb{N} \times \mathbb{N} \to \mathbb{N}$
$multacc(\langle 0, y, a \rangle) = a$ $\langle 2 \rangle$
$multacc(\langle x + 1, y, a \rangle) = multacc(\langle x, y, a + y \rangle)$ $\langle 3 \rangle$

Observações:

- *multacc* é uma função auxiliar que só é conhecida por *mult* e não pode ser usada diretamente;
- *mult* é indiretamente recursiva;
- as variáveis são locais às equações;
- as linhas <2> e <3> são mutuamente exclusivas;

4. Tipos de recursão

Uma função *f* é dita <u>recursiva na cauda</u> se os resultados de quaisquer chamadas de *f* são usados imediatamente como resultado de *f*, sem quaisquer computações adicionais. Desta forma, em uma definição recursiva na cauda a recursão é usada simplesmente para chamar a "mesma função" só que com argumentos diferentes.
As funções recursivas na cauda são uma subclasse das funções recursivas normais. O uso de funções recursivas na cauda vem do fato de que elas podem ser traduzidas diretamente para laços de repetição em algoritmos descritos no paradigma imperativo.

Exemplo: algoritmo para calcular a função potência

Versão recursiva convencional	Versão recursiva na cauda
$pot: \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ $pot(\langle b, p\rangle) = 1, se\ p = 0$ $pot(\langle b, p\rangle) = b \times pot(\langle b, p-1\rangle), se\ p > 0$	$pot: \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ $pot(\langle b, p\rangle) = potAux(\langle b, p, 1\rangle)$ onde $potAux: \mathbb{N} \times \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ $potAux(\langle b, p, a\rangle) = a, se\ p = 0$ $potAux(\langle b, p, a\rangle) = potAux(\langle b, p-1, a * b\rangle), se\ p > 0$

5. Computação como uma sequência de estados

Até agora vimos que um procedimento pode ser descrito como uma sequência de passos. Durante a execução desta sequência, o <u>estado da computação</u> é definido pelo conjunto dos valores de suas variáveis. Por exemplo, no algoritmo:

$mult: \mathbb{N} \times \mathbb{N} \to \mathbb{N}$
$mult(\langle m, n\rangle) = multacc(\langle m, n, 0\rangle)$ (1)
onde
$multacc: \mathbb{N} \times \mathbb{N} \times \mathbb{N} \to \mathbb{N}$
$multacc(\langle 0, y, a\rangle) = a$ (2)
$multacc(\langle x+1, y, a\rangle) = multacc(\langle x, y, a+y\rangle)$ (3)

Podemos determinar o estado da computação, a qualquer momento, consultando o valor das variáveis *x*, *y* e *a* como segue (ao lado de cada chamada está indicada a regra aplicada e a forma como os valores foram substituídos):

mult(5,3) =	multacc(5,3,0)	$(1, [m \to 5, n \to 3])$
=	multacc(4,3,3)	$(3, [x \to 5, y \to 3, a \to 0])$
=	multacc(3,3,6)	$(3, [x \to 4, y \to 3, a \to 3])$
=	multacc(2,3,9)	$(3, [x \to 3, y \to 3, a \to 6])$
=	multacc(1,3,12)	$(3, [x \to 2, y \to 3, a \to 9])$
=	multacc(0,3,15)	$(3, [x \to 1, y \to 3, a \to 12])$
=	15	$(2, [x \to 0, y \to 3, a \to 15])$

Uma forma equivalente de descrever o algoritmo *multacc* é através da descrição da mudança dos valores das variáveis, que normalmente é chamada de <u>estado da computação</u>. Neste caso, o estado pode ser representado por uma tripla $s: \mathbb{N} \times \mathbb{N} \times \mathbb{N} \triangleq \langle x, y, a \rangle$.

Assim, executando o algoritmo $mult$ com os valores 5 e 3 teremos como resultado a seguinte sequência de estados para $multacc$ (note que as tuplas de valores que representam os estados da computação estão indicadas entre "⟨ ⟩" e os sucessivos estados separados por "→"):

$$\langle 5,3,0 \rangle \rightarrow \langle 4,3,3 \rangle \rightarrow \langle 3,3,6 \rangle \rightarrow \langle 2,3,9 \rangle \rightarrow \langle 1,3,12 \rangle \rightarrow \langle 0,3,15 \rangle$$

Assumindo que $s_0 = \langle 5,3,0 \rangle$ é o estado inicial e $s_5 = \langle 0,3,15 \rangle$ é o estado final da computação desejada, é possível observar que a transição de estado é definida pela regra:

$$\langle x, y, a \rangle \rightarrow \langle x - 1, y, a + y \rangle$$

Estados	**Variáveis**
$s_0 = \langle x, y, 0 \rangle$	$\langle x_0, y_0, a_0 \rangle$
$s_1 = \langle x - 1, y, y \rangle$	$\langle x_1, y_1, a_1 \rangle$
$s_2 = \langle x - 2, y, y + y \rangle$	$\langle x_2, y_2, a_2 \rangle$
...	...
$s_k = \langle 0, y, {}_0^k\Sigma y_k \rangle$	$\langle x_k, y_k, a_k \rangle$

O estado s_0 representa o estado inicial da computação e o estado s_k, o estado final. Para cada $j = 0 \ldots k, s_j = \langle x - j, y, {}_0^j\Sigma y_j \rangle$ é o estado após o *j*-ésimo passo de computação.

Note que essa sequência de estados é resultado da aplicação das regras na seguinte sequência:

$$(3) \rightarrow (3) \rightarrow (3) \rightarrow (3) \rightarrow (3) \rightarrow (3) \rightarrow (2)$$

A aplicação recursiva da regra (3) denota um <u>laço</u> ou repetição de instruções que irá ocorrer até que a condição de parada da regra (2) ocorra. Cada aplicação da regra (3) corresponde a uma <u>iteração</u> do laço do algoritmo descrito anteriormente.

Note que analisando o estado da computação a cada iteração do laço conseguimos definir uma asserção (uma afirmação) a respeito da evolução destes estados durante as sucessivas iterações. No caso é possível deduzir que:

$$m \times n = x_k \times y_k + a_k, k \in \mathbb{N}$$

Onde temos os valores iniciais de *m* e *n*, e x_k, y_k e a_k são os valores das variáveis após a iteração *k*, isto é, após $k \in \mathbb{N}$ repetições do laço. Esta asserção é verdadeira no início quando $x = 5, y = 3, a = 0$ e continua sendo verdadeira durante todas as iterações do laço. Dito de outra forma, esta asserção continua sendo **invariavelmente verdadeira** durante todas

as iterações e ainda será verdadeira após a última iteração. Esta **asserção invariante** expressa a essência deste procedimento computacional (algoritmo) e também é chamada de invariante de laço, ou nesse caso, de invariante da recursão.

No exemplo apresentado o invariante se confunde com a própria chamada recursiva, porém, nem sempre é tão direto.

Observe a tabela a seguir:

Estados	**Invariante**
$s_0 = \langle 5,3,0 \rangle = \langle x_0, y_0, a_0 \rangle$	$5 \times 3 = 5 \times 3 + 0$
$s_1 = \langle 4,3,3 \rangle = \langle x_1, y_1, a_1 \rangle$	$5 \times 3 = 4 \times 3 + 3$
$s_2 = \langle 3,3,6 \rangle = \langle x_2, y_2, a_2 \rangle$	$5 \times 3 = 3 \times 3 + 6$
$s_3 = \langle 2,3,9 \rangle = \langle x_3, y_3, a_3 \rangle$	$5 \times 3 = 2 \times 3 + 9$
$s_4 = \langle 1,3,12 \rangle = \langle x_4, y_4, a_4 \rangle$	$5 \times 3 = 1 \times 3 + 12$
$s_5 = \langle 0,3,15 \rangle = \langle x_5, y_5, a_5 \rangle$	$5 \times 3 = 0 \times 3 + 15$

Entender um algoritmo como uma sequência de estados não apenas facilita a compreensão da ideia de invariante como permite uma visão diferente da tradicional (sequência de passos). Dispor de duas formas para analisar um mesmo objeto fornece mais ferramentas para tratá-lo e uma compreensão mais profunda do mesmo.

Pode-se definir outras invariantes para o algoritmo de multiplicação. Por exemplo:

- $\forall k: \mathbb{N}. x_k \geq 0 \wedge y_k \geq 0$
- $\forall k: \mathbb{N}. x_0 \geq y_k$
- $\forall k: \mathbb{N}. x_k \geq x_{k+1}$

Muitos asserções são invariante para uma computação em particular. Algumas ajudam a entender o problema, outras não.

Diz-se que uma **invariante é adequada** se e somente se ela é uma proposição que expressa uma relação em termos do que pretendemos com uma determinada computação, isto é, se a invariante tem a forma

$$G = E$$

Onde *G* é o objetivo (do inglês *goal*) da computação e *E* é uma expressão que usa valores do estado corrente.

No caso do algoritmo da multiplicação temos:

$G \triangleq m \times n$

$E \triangleq x_k \times y_k + a_k, k \in \mathbb{N}$

6. Máquina de estados

Uma máquina de estados é um modelo abstrato adequado para um algoritmo cuja computação se dá passo a passo. A seguir será definido mais formalmente a noção apresentada no capítulo anterior.

Formalmente, uma máquina de estados é definida por:

- Um conjunto de estados S;
- Uma relação de transição entre estados dada por uma relação binária $\rightarrow \subseteq S \times S$;
- Um estado inicial $s_0 \in S$.

A execução de um algoritmo/programa é representada pela máquina de estado onde, a partir de um estado inicial, os comandos do programa induzem as transições de estado da máquina. A sequência de estados obtidos a partir do estado inicial é um modelo que representa a computação realizada.

De maneira mais formal, uma execução de uma máquina de estados (ou traço, do inglês *trace*) é uma sequência (possivelmente infinita) de estados tais que:

- Começa com o estado inicial s_0;
- Se s e s' são estados consecutivos na sequência, então existe uma transição $s \rightarrow s'$ entre os dois estados.

Por fim, diz-se que um estado é alcançável se ele aparece em uma sequência que corresponde a uma possível execução da máquina de estados.

7. Completando a especificação formal de algoritmos

Observe o algoritmo que segue que decrementa um número natural de uma unidade:

$$\begin{array}{l} decNat: \mathbb{N} \rightarrow \mathbb{N} \\ decNat(m) = m - 1 \end{array}$$

O que acontece se aplicamos essa função sobre o valor zero?

Note que a função *decNat* é uma função parcial que não está definida para o valor zero. Se for aplicada com este valor irá cair em um estado de indefinição.

Existem duas formas usuais de contornar este problema:

a) Prever todos os casos no corpo da função:

$$\begin{array}{l} decNat: \mathbb{N} \rightarrow \mathbb{N} \\ decNat(0) = 0 \\ decNat(m) = m - 1, se\ m > 0 \end{array}$$

b) Especificar formalmente quais são as condições nas quais a função se aplica, isto é, quais são as pré-condições de aplicação da função:

$$\begin{array}{l} decNat: \mathbb{N} \rightarrow \mathbb{N} \\ \ requer: m \geq 1 \\ decNat(m) = m - 1 \end{array}$$

Note que neste exemplo em particular o custo de especificar a pré-condição é o mesmo de prever todas as possibilidades no corpo da função. Em situações mais complexas, porém, não apenas é mais simples especificar as pré-condições de aplicação da função como a especificação das mesmas as torna mais explícitas e fáceis de verificar. Desta forma a especificação das pré-condições de aplicação de uma função é uma boa prática que deve ser observada sempre.

Outra boa prática é a especificação das pós-condições, isto é, qual o resultado esperado a partir da aplicação da função. Além de explicitar o objetivo do algoritmo, facilita a comprovação de que os resultados são alcançados.

Especificando-se as pré e pós condições nossa função *decNat* fica da seguinte forma:

$$\begin{array}{l} decNat: \mathbb{N} \rightarrow \mathbb{N} \\ \ requer: m \geq 1 \\ \ garante: decNat(x) = x - 1 \\ decNat(m) = m - 1 \end{array}$$

8. Conclusão

Nosso objetivo foi definir uma maneira de formalizar a noção de especificação de algoritmo de maneira a permitir, futuramente, a análise e prova de propriedades (tal como a correção e terminação) sobre estes algoritmos. Além disso, o conceito de correção de um algoritmo sempre será uma noção relativa, ou seja, um algoritmo é correto ou não sempre em relação à sua especificação.

Uma pré-condição é uma asserção sobre a entrada que o usuário do algoritmo ou função deve respeitar. A pré-condição é uma asserção que deve ser sempre ser verdadeira antes da execução do algoritmo ou avaliação da função. Se esta condição for violada, o efeito do algoritmo ou função torna-se indefinido e, portanto, pode ou não executar aquilo para o qual foi projetado.

Uma pós-condição é uma asserção sobre a saída do algoritmo ou função que é garantida pelo programador (ou quem projetou o algoritmo ou função). A pós-condição também é uma asserção que deve ser sempre verdadeira após a execução do algoritmo.

Uma invariante é também uma asserção lógica que deve ser sempre verdadeira durante uma certa fase da execução do algoritmo (nos casos apresentados ela é verdadeira antes e

depois de um determinado algoritmo). Veremos na sequência que invariantes são fundamentais para a demonstração da correção de algoritmos.